



Engineering at Anthropic



An update on recent Claude Code quality reports

We traced recent reports of Claude Code quality issues to three separate changes. Here's what happened and what we're changing.

Published Apr 23, 2026

Over the past month, we've been looking into reports that Claude's responses have worsened for some users. We've traced these reports to three separate changes that affected Claude Code, the Claude Agent SDK, and Claude Cowork. The API was not impacted.

All three issues have now been resolved as of April 20 (v2.1.116).

In this post, we explain what we found, what we fixed, and what we'll do differently to ensure similar issues are much less likely to happen again.

We take reports about degradation very seriously. We never intentionally degrade our models, and we were able to immediately confirm that our API and inference layer were unaffected.

After investigation, we identified three different issues:

1. On March 4, we changed Claude Code's default reasoning effort from `high` to `medium` to reduce the very long latency—enough to make the UI appear frozen—some users were seeing in `high` mode. This was the wrong tradeoff. We reverted this change on April 7 after users told us they'd prefer to default to higher intelligence and opt into lower effort for simple tasks. This impacted Sonnet 4.6 and Opus 4.6.
2. On March 26, we shipped a change to clear Claude's older thinking from sessions that had been idle for over an hour, to reduce latency when users resumed those sessions. A bug caused this to keep happening every turn for the rest of the session instead of just once, which made Claude seem forgetful and repetitive. We fixed it on April 10. This affected Sonnet 4.6 and Opus 4.6.
3. On April 16, we added a system prompt instruction to reduce verbosity. In combination with other prompt changes, it hurt coding quality and was reverted on April 20. This impacted Sonnet 4.6, Opus 4.6, and Opus 4.7.

Because each change affected a different slice of traffic on a different schedule, the aggregate effect looked like broad, inconsistent degradation. While we began investigating reports in early March, they were challenging to distinguish from normal variation in user feedback at first, and neither our internal usage nor evals initially reproduced the issues identified.

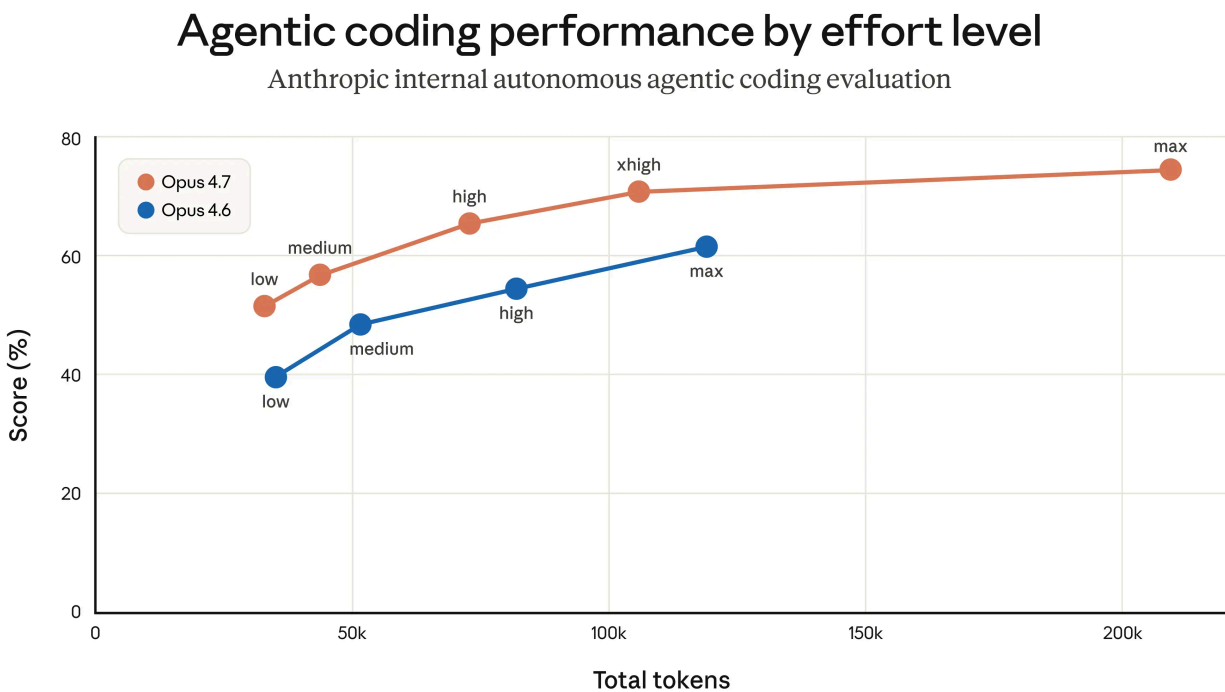
This isn't the experience users should expect from Claude Code. As of April 23, we're resetting usage limits for all subscribers.

A change to Claude Code's default reasoning effort

When we released Opus 4.6 in Claude Code in February, we set the default reasoning effort to `high`.

Soon after, we received user feedback that Claude Opus 4.6 in high effort mode would occasionally think for too long, causing the UI to appear frozen and leading to disproportionate latency and token usage for those users.

In general, the longer the model thinks, the better the output. Effort levels are how Claude Code lets users set that tradeoff—more thinking versus lower latency and fewer usage limit hits. As we calibrate effort levels for our models, we take this tradeoff into account in order to pick points along the test-time-compute curve that give people the best range of options. In the product layer, we then choose which point along this curve we set as our default, and that is the value we send to the Messages API as the effort parameter; we then make the other options available via `/effort`.



In our internal evals and testing, medium effort achieved slightly lower intelligence with significantly less latency for the majority of tasks. It also didn't suffer from the same issues with occasional very long tail latencies for thinking, and it helped maximize users' usage limits. As a result, we rolled out a change making medium the default effort, and explained the rationale via in-product dialog.



Claude Code v2.1.111
Opus 4.7 (1M context) · API Usage Billing
/private/tmp/cc-sandbox-ecallout

? for shortcuts · /issue to report model behavior issues

We recommend medium effort for Opus

Effort determines how long Claude thinks for when completing your task. We recommend medium effort for most tasks to balance speed and intelligence and maximize rate limits. Use ultrathink to trigger high effort when needed.

☐ low · ☒ medium · ☐ high

- > 1. ☒ Medium (recommended)
- 2. ☐ High
- 3. ☐ Low

Soon after rolling out, users began reporting that Claude Code felt less intelligent. We shipped a number of design iterations to make the current effort setting clearer in order to alert people they could change the default (notices on startup, an inline effort selector, and bringing back ultrathink), but most users retained the medium effort default.

After hearing feedback from more customers, we reversed this decision on April 7. All users now default to **xhigh** effort for Opus 4.7, and **high** effort for all other models.

A caching optimization that dropped prior reasoning

When Claude reasons through a task, that reasoning is normally kept in the conversation history so that on every subsequent turn, Claude can see why it made the edits and tool calls it did.

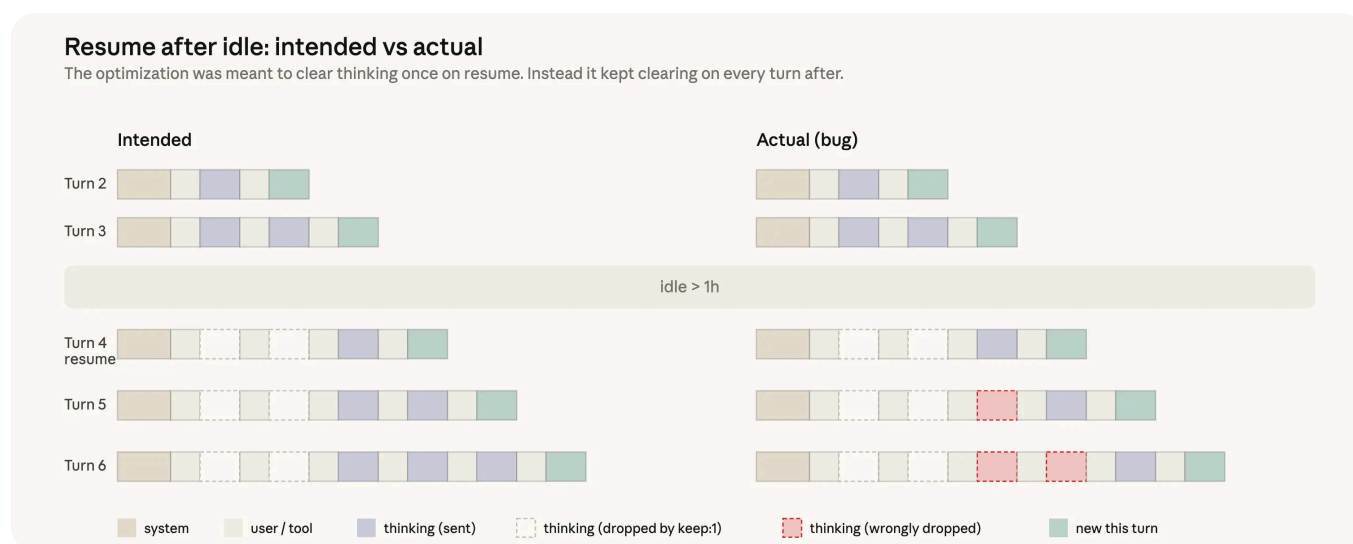
On March 26, we shipped what was meant to be an efficiency improvement to this feature. We use prompt caching to make back-to-back API calls cheaper and faster for users. Claude writes the input tokens to the cache when it makes an API request, then after a period of inactivity the prompt is evicted from cache, making room for other

prompts. Cache utilization is something we manage carefully (more on our [approach](#)).

The design should have been simple: if a session has been idle for more than an hour, we could reduce users' cost of resuming that session by clearing old thinking sections. Since the request would be a cache miss anyway, we could prune unnecessary messages from the request to reduce the number of uncached tokens sent to the API. We'd then resume sending full reasoning history. To do this we used the `clear_thinking_20251015` API header along with `keep:1`.

The implementation had a bug. Instead of clearing thinking history once, it cleared it on every turn for the rest of the session. After a session crossed the idle threshold once, each request for the rest of that process told the API to keep only the most recent block of reasoning and discard everything before it. This compounded: if you sent a follow-up message while Claude was in the middle of a tool use, that started a new turn under the broken flag, so even the reasoning from the current turn was dropped. Claude would continue executing, but increasingly without memory of why it had chosen to do what it was doing. This surfaced as the forgetfulness, repetition, and odd tool choices people reported.

Because this would continuously drop thinking blocks from subsequent requests, those requests also resulted in cache misses. We believe this is what drove the separate reports of usage limits draining faster than expected.



Two unrelated experiments made it challenging for us to reproduce the issue at first: an internal-only server-side experiment related to message queuing; and an

orthogonal change in how we display thinking suppressed this bug in most CLI sessions, so we didn't catch it even when testing external builds.

This bug was at the intersection of Claude Code's context management, the Anthropic API, and extended thinking. The changes it introduced made it past multiple human and automated code reviews, as well as unit tests, end-to-end tests, automated verification, and dogfooding. Combined with this only happening in a corner case (stale sessions) and the difficulty of reproducing the issue, it took us over a week to discover and confirm the root cause.

As part of the investigation, we back-tested Code Review against the offending pull requests using Opus 4.7. When provided the code repositories necessary to gather complete context, Opus 4.7 found the bug, while Opus 4.6 didn't. To prevent this from happening again, we are now landing support for additional repositories as context for code reviews.

We fixed this bug on April 10 in v2.1.101.

A system prompt change to reduce verbosity

Our latest model, Claude Opus 4.7, has a notable behavioral quirk relative to its predecessor: as we wrote about at launch, it tends to be quite verbose. This makes it smarter on hard problems, but it also produces more output tokens.

A few weeks before we released Opus 4.7, we started tuning Claude Code in preparation. Each model behaves slightly differently, and we spend time before each release optimizing the harness and product for it.

We have a number of tools to reduce verbosity: model training, prompting, and improving thinking UX in the product. Ultimately we used all of these, but one addition to the system prompt caused an outsized effect on intelligence in Claude Code:

“Length limits: keep text between tool calls to ≤ 25 words. Keep final responses to ≤ 100 words unless the task requires more detail.”

After multiple weeks of internal testing and no regressions in the set of evaluations we ran, we felt confident about the change and shipped it alongside Opus 4.7 on April 16.

As part of this investigation, we ran more ablations (removing lines from the system prompt to understand the impact of each line) using a broader set of evaluations. One of these evaluations showed a 3% drop for both Opus 4.6 and 4.7. We immediately reverted the prompt as part of the April 20 release.

Going forward

We are going to do several things differently to avoid these issues: we'll ensure that a larger share of internal staff use the exact public build of Claude Code (as opposed to the version we use to test new features); and we'll make improvements to our Code Review tool that we use internally, and ship this improved version to customers.

We're also adding tighter controls on system prompt changes. We will run a broad suite of per-model evals for every system prompt change to Claude Code, continuing ablations to understand the impact of each line, and we have built new tooling to make prompt changes easier to review and audit. We've additionally added guidance to our CLAUDE.md to ensure model-specific changes are gated to the specific model they're targeting. For any change that could trade off against intelligence, we'll add soak periods, a broader eval suite, and gradual rollouts so we catch issues earlier.

We recently created @ClaudeDevs on X to give us the room to explain product decisions and the reasoning behind them in depth. We'll share the same updates in centralized threads on GitHub.

Finally, we'd like to thank our users: the people who used the `/feedback` command to share their issues with us (or who posted specific, reproducible examples online) are the ones who ultimately allowed us to identify and fix these problems. Today we are resetting usage limits for all subscribers.

We're immensely grateful for your feedback and for your patience.

Get the developer newsletter

Product updates, how-tos, community spotlights, and more.
Delivered monthly to your inbox.

Enter your email



Please provide your email address if you'd like to receive our monthly developer newsletter. You can unsubscribe at any time.



Products

[Claude](#)

[Claude Code](#)

[Claude Code Enterprise](#)

[Claude Cowork](#)

[@Claude](#)

[Claude Design](#)

[Claude Science](#)

[Claude Security](#)

[Claude in Chrome](#)

[Claude for Microsoft 365](#)

[Skills](#)

[Download app](#)

[Pricing](#)

[Log in to Claude](#)

Models

[Mythos](#)

[Fable](#)

Opus

Sonnet

Haiku

Solutions

AI agents

Code modernization

Coding

Customer support

Cybersecurity

Enterprise

Financial services

Government

Healthcare

Higher education

K-12 teachers

Legal

Life sciences

Nonprofits

Small business

Claude Platform

Overview

Developer docs

Pricing

Ecosystem

Marketplace

Regional compliance

Claude on AWS

Google Cloud

[Microsoft Foundry](#)

[Console login](#)

Resources

[Blog](#)

[Claude partner network](#)

[Community](#)

[Connectors](#)

[Courses](#)

[Customer stories](#)

[Engineering at Anthropic](#)

[Events](#)

[Plugins](#)

[Powered by Claude](#)

[Service partners](#)

[Tutorials](#)

[Use cases](#)

Programs

[Startups](#)

[Research Labs](#)

Help and security

[Availability](#)

[Status](#)

[Support center](#)

Company

[Anthropic](#)

[Careers](#)

[Leadership](#)

[Policy](#)

[Economic Futures](#)

[Research](#)

[News](#)

[Claude's Constitution](#)

[Claude Corps](#)

[Keep thinking](#)

[Policy on the AI Exponential](#)

[Responsible Scaling Policy](#)

[Security and compliance](#)

[Transparency](#)

Terms and policies

[Privacy choices](#)

[Privacy policy](#)

[Consumer health data privacy policy](#)

[Responsible disclosure policy](#)

[Terms of service: Commercial](#)

[Terms of service: Consumer](#)

[Terms of Service: US K-12](#)

[Data Processing Agreement: US K-12](#)

[Usage policy](#)

© 2026 Anthropic PBC

